UNIVERSITY OF CALIFORNIA, MERCED

CSE179: Parallel Computing

March 2

Dong Li

Spring 2026

UCMERCED

www.ucmerced.edu

Lecture Topics

- MPI
  - Principles
  - Building blocks
  - Examples

Message Passing Overview

- The logical view of a message-passing platform
  - *p* processes
  - each with its own exclusive address space
- All data must be explicitly partitioned and placed
- All interactions (read-only or read/write) are typically two-sided
  - process that has the data
  - process that wants to access the data
- Typically use single program multiple data (SPMD) model
- The bottom line …
  - Strengths
    - simple performance model: underlying costs are explicit
    - portable high performance
  - Weakness: two-sided model can be awkward to program

Send and Receive

- Prototype operations
  - send(void *sendbuf, int nelems, int dest_rank)
  - receive(void *recvbuf, int nelems, int source_rank)
- Consider the following code fragments:

```
Processor 0
a = 100;
send(&a, 1, 1);
a = 0;
```

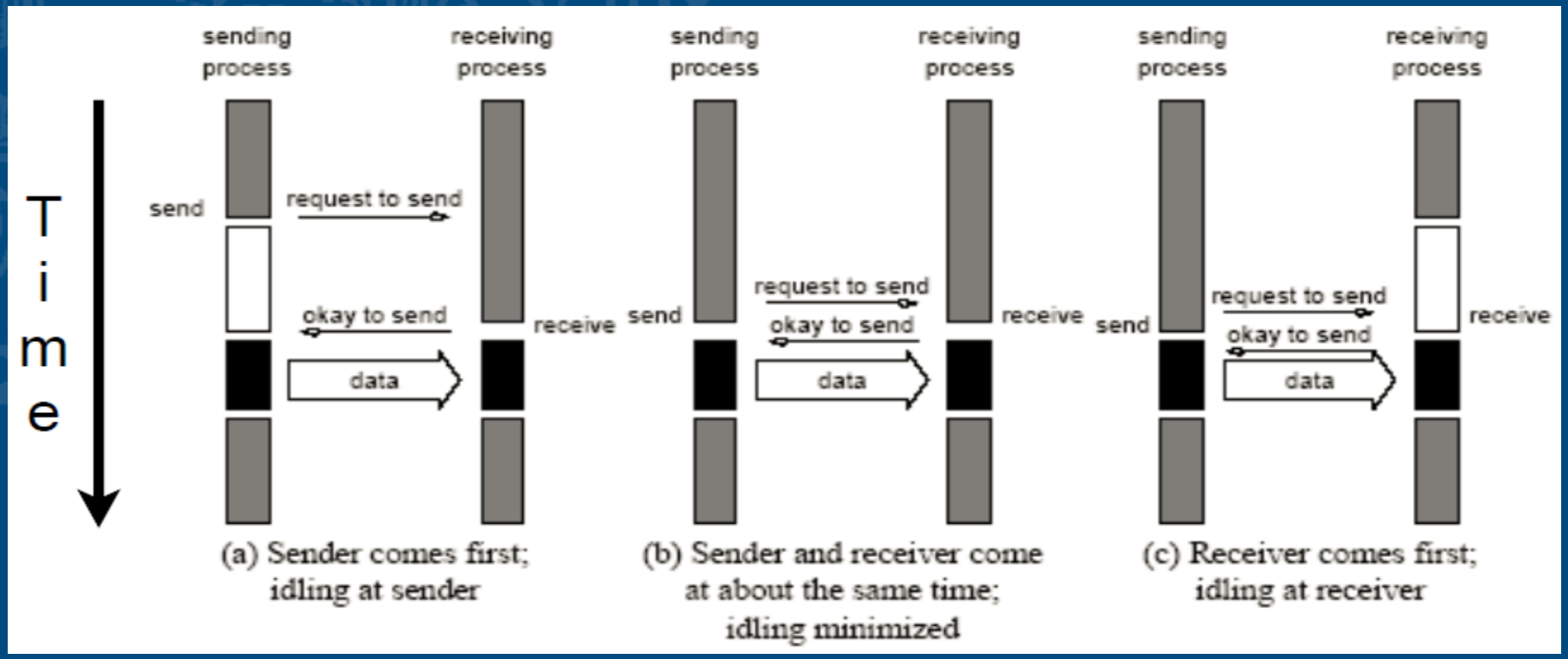
```
Processor 1
receive(&a, 1, 0)
printf("%d\n", a);
```

- The semantics of send
  - value received by process P1 must be 100, not 0
  - motivates the design of send and receive protocols

Blocking Message Passing

- Non-buffered, blocking sends
  - send does not return until the matching receive executes
  - Issues
    - Idling
    - deadlock
- Buffered, blocking sends
  - sender copies the data into the designated buffer
  - returns after the copy completes
  - data is copied into buffer at the receiver as well
  - avoids idling at the expense of copying

Non-Buffered, Blocking Message Passing



- Handshaking for blocking non-buffered send/receive
- Idling occurs when operations are not simultaneous

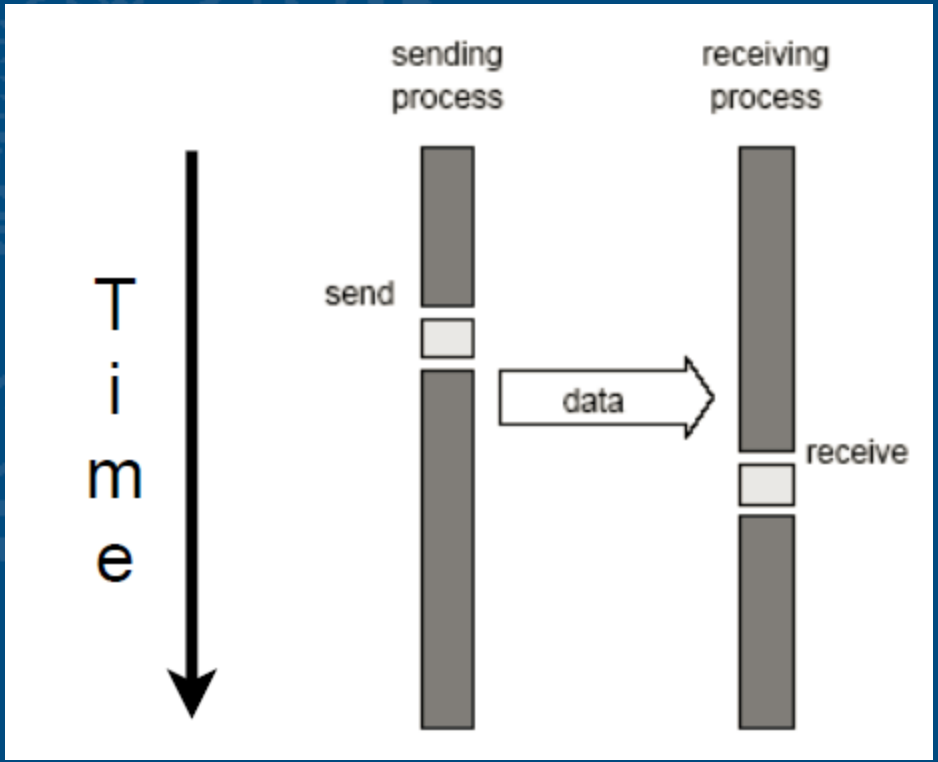
(Case shown: no NIC support for communication)

UCMERCED

Buffered, Blocking Message Passing

- Buffering at the sending and receiving ends avoids idling
- Sender copies the data into the buffer and returns
- Data is buffered at the receiver as well
- Tradeoff
  - bufferring trades idling overhead for data copying overhead

Buffered, Blocking Message Passing



- NIC moves the data behind the scenes

(illustrations show case when sender comes first)

Buffered Blocking Message Passing

- Bounded buffer sizes can have significant impact on performance

Processor 0

```
for (i = 0; i < 1000; i++){
    produce_data(&a);
    send(&a, 1, 1);
 }
```

Processor 1

```
for (i = 0; i < 1000; i++){
    receive(&a, 1, 0);
    consume_data(&a);
}
```

- Larger buffers enable the computation to tolerate asynchrony better

UCMERCED

Buffered, Blocking Message Passing

- Deadlocks are possible with buffering since receive operations block

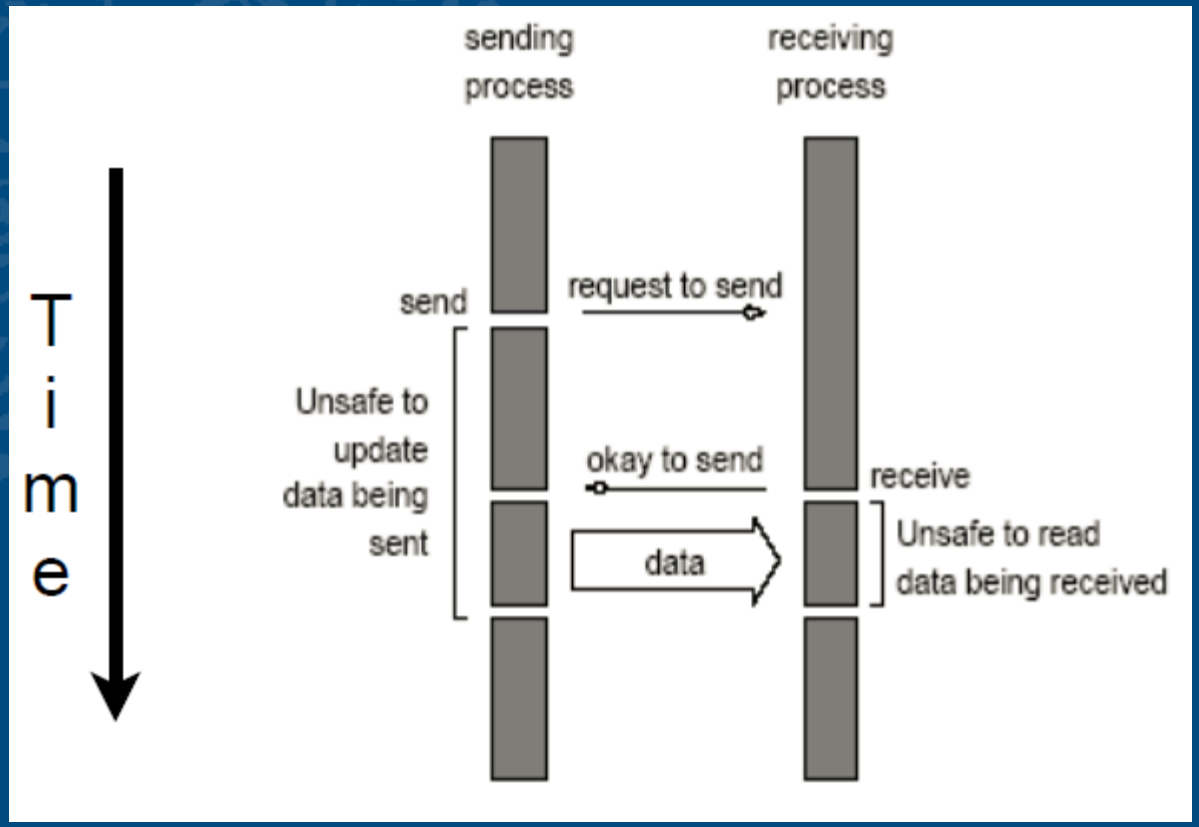
```
Processor 0
receive(&a, 1, 1);
send(&b, 1, 1);
```

```
Processor 1
receive(&a, 1, 0);
send(&b, 1, 0);
```

Non-Blocking Message Passing

- Non-blocking protocols
  - send and receive return before it is
    - sender: data can be overwritten before it is sent
    - receiver: can read data out of buffer before it is received
  - ensuring proper usage is the programmer's responsibility
  - status check operation to ascertain safe completion
- Benefit
  - capable of overlapping communication with useful computation
- Typical implementations
  - provide both blocking and non-blocking primitives

Non-Blocking Message Passing



- NIC moves the data behind the scenes

MPI: the Message Passing Interface

- Standard library for message-passing
  - Portable
  - almost ubiquitously available
  - high performance
  - C and Fortran APIs
- MPI standard defines
  - syntax of library routines
  - semantics of library routines
- Details
  - MPI routines, data-types, and constants are prefixed by “MPI_”
- Simple to get started
  - fully-functional programs using only six library routines

Scope of the MPI Standards

- Communication contexts
- Datatypes
- Point-to-point communication
- Collective communication (synchronous, non-blocking)
- Process groups
- Process topologies
- Environmental management and inquiry
- The Info object
- Process creation and management
- One-sided communication (refined for MPI-3)
- External interfaces
- Parallel I/O
- Language bindings for Fortran, C and C++
- Profiling interface (PMPI)

MPI: the Message Passing Interface

- Minimal set of MPI routines

- MPI_Init
  - initialize MPI
- MPI_Finalize
  - terminate MPI
- MPI_Comm_size
  - determine the number of processes in group
- MPI_Comm_rank
  - determine id of calling process in group
- MPI_Send
  - send message
- MPI_Recv
  - receive message

Starting and Terminating the MPI Programs

- int MPI_Init(int *argc, char ***argv)
  - initialization: must call this prior to other MPI routines
  - Effects
    - strips off and processes any MPI command-line arguments
    - initializes MPI environment
- int MPI_Finalize()
  - must call at the end of the computation
  - Effect
    - performs various clean-up tasks to terminate MPI environment
- Return codes
  - MPI_SUCCESS
  - MPI_ERROR

Communicators

- MPI_Comm: communicator = communication domain
  - group of processes that can communicate with one another
- Supplied as an argument to all MPI message transfer routines
- Process can belong to multiple communication domains
  - domains may overlap
- MPI_COMM_WORLD: root communicator
  - includes all the processes

Communicator Inquiry Functions

- int MPI_Comm_size(MPI_Comm comm, int *size)
  - determine the number of processes
- int MPI_Comm_rank(MPI_Comm comm, int *rank)
  - index of the calling process
  - 0 ≤ rank < communicator size

“Hello World” Using MPI

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
  int npes, myrank;
  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &npes);
  MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
  printf("From process %d out of %d, Hello World!\n",
           myrank, npes);
  MPI_Finalize();
  return 0;
}
```

UCMERCED

Sending and Receiving Messages

- int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest_pe, int tag, MPI_Comm comm)
- int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source_pe, int tag, MPI_Comm comm, MPI_Status *status)

- Message source or destination PE
  - index of process in the communicator comm
  - receiver wildcard: MPI_ANY_SOURCE
    - any process in the communicator can be source
- Message-tag: integer values, 0 ≤ tag < MPI_TAG_UB
  - receiver tag wildcard: MPI_ANY_TAG
- Receiver constraint
  - message size ≤ buffer length specified

MPI Primitive Data Types

MPI data type	C data type
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	8 bits
MPI_PACKED	packed sequence of bytes

Receiver Status Inquiry

- MPI_Status
  - stores information about an MPI_Recv operation
  - data structure

```
typedef struct MPI_Status {
        int MPI_SOURCE;
        int MPI_TAG;
        int MPI_ERROR; };
```

- int MPI_Get_count(MPI_Status *status, MPI_Datatype, datatype, int *count)
  - returns the count of data items received
  - not directly accessible from status variable

UCMERCED

Deadlock Pitfalls

```
int a[10], b[10], myrank;
MPI_Status s1, s2;
...
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
if (myrank == 0) {
    MPI_Send(a, 10, MPI_INT, 1, 1, MPI_COMM_WORLD);
    MPI_Send(b, 10, MPI_INT, 1, 2, MPI_COMM_WORLD);
}
else if (myrank == 1) {
    MPI_Recv(b, 10, MPI_INT, 0, 2, MPI_COMM_WORLD, &s1);
    MPI_Recv(a, 10, MPI_INT, 0, 1, MPI_COMM_WORLD, &s2);
}
...
```

destination

tag

- Deadlock if MPI_Send is blocking

More Deadlock Pitfalls

- Send data to neighbor to your right on a ring ...

```
int a[10], b[10], npes, myrank;
MPI_Status status;
...
MPI_Comm_size(MPI_COMM_WORLD, &npes);

MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1,
         MPI_COMM_WORLD);

MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1,
         MPI_COMM_WORLD, &status);
...
```

- Deadlock if MPI_Send is blocking

Avoiding Deadlock

- Send data to neighbor to your right on a ring ...
  - Break the circular wait

```
int a[10], b[10], npes, myrank;
MPI_Status status;
...
MPI_Comm_size(MPI_COMM_WORLD, &npes);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

if (myrank%2 == 1) {   // odd processes send first, receive second
        MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1,
                        MPI_COMM_WORLD);
        MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1,
                        MPI_COMM_WORLD, &status);
}
else {                  // even processes receive first, send second
        MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1,
                        MPI_COMM_WORLD, &status);
        MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1,
                        MPI_COMM_WORLD);
}
...
```

Primitives for Non-blocking Communication

- Non-blocking send and receive return before they complete

```
int MPI_Isend(void *buf, int count, MPI_Datatype datatype, int dest,
        int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Irecv(void *buf, int count, MPI_Datatype datatype, int
        source, int tag, MPI_Comm comm,
        MPI_Request *request)
```

- MPI_Test: has a particular non-blocking request finished?

```
int MPI_Test(MPI_Request *request, int *flag,
MPI_Status *status)
```

Primitives for Non-blocking Communication

- MPI_Waitany: block until some request in a set completes

```
int MPI_Wait_any(int req_cnt, MPI_Request  *req_array,
                 int *req_index, MPI_Status *status)
```

- MPI_Wait: block until a particular request completes

```
int MPI_Wait(MPI_Request *request, MPI_Status *status)
```

Avoiding Deadlocks with NB Primitives

- Using non-blocking operations avoids most deadlocks

```
int a[10], b[10], myrank;
MPI_Request r1, r2;
...
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
if (myrank == 0) {
   MPI_ISend(a, 10, MPI_INT, 1, 1, MPI_COMM_WORLD, &r1);
   MPI_ISend(b, 10, MPI_INT, 1, 2, MPI_COMM_WORLD, &r2);
}
else if (myrank == 1) {
   MPI_IRecv(b, 10, MPI_INT, 0, 2, &status, MPI_COMM_WORLD, &r1);
   MPI_IRecv(a, 10, MPI_INT, 0, 1, &status, MPI_COMM_WORLD, &r2);
}
...
```

tag

Overlapping Communication Example

- Original
  - for each element
    - Compute element
  - communicate boundary layer to partner with blocking send
  - receive from partner
  - compute with data from partner
- Optimized
  - for each block
    - for each element in block
      - Compute each element
    - communicate block to partner with non-blocking send
  - for each block
    - wait for block from partner
    - compute with block from partner

Message Exchange

- To exchange messages in a single call (both send and receive)

```
int MPI_Sendrecv(void *sendbuf, int sendcount,
        MPI_Datatype senddatatype, int dest, int
        sendtag, void *recvbuf, int recvcount,
        MPI_Datatype recvdatatype, int source, int recvtag,
        MPI_Comm comm, MPI_Status *status)
```

- Requires both send and receive arguments
- To use same buffer for both send and receive

```
int MPI_Sendrecv_replace(void *buf, int count,
        MPI_Datatype datatype, int dest, int sendtag,
        int source, int recvtag, MPI_Comm comm,
        MPI_Status *status)
```

Reading Materials

- Chapter 6
  - 6.1-6.3

UCMERCED